

Data Handling: Import, Cleaning and Visualisation

Additional notes for Exercise/Workshop 2: Computer Code and Data Storage

Aurélien Sallin

06/10/2023

Repetition: Data Structures in R

R provides different object classes to work with data. In simple terms, they differ with regard to how data is structured when saved in a variable. This, in turn, defines how we can access different parts of the data in this variable. For example, if we store a bunch of numbers in a numeric vector, we can then access each of these numbers individually, by telling R to give us the first, second, nth element of this vector. If we store data in a two-dimensional structure such as a matrix, we can then access the data column-wise, row-wise, or cell-wise. Depending on the task, one or the other object class is preferable to store and work with data.

Vectors and Lists

```
# A vector containing numeric (or integer) values
```

```
numeric_vector <- 10:20
```

```
numeric_vector[2]
```

```
## [1] 11
```

```
numeric_vector[2:5]
```

```
## [1] 11 12 13 14
```

```
# A string vector ('a vector containing text')
```

```
string_vector <- c("a", "b", "c")
```

```
string_vector[-3]
```

```
## [1] "a" "b"
```

```
# Lists
```

```
# A list can contain different types of elements, for example a numeric vector and a string_vector
```

```
mylist <- list(numbers = numeric_vector, letters = string_vector)
```

```
mylist
```

```
## $numbers
```

```
## [1] 10 11 12 13 14 15 16 17 18 19 20
```

```
##
```

```
## $letters
```

```
## [1] "a" "b" "c"
```

```
# We can access the elements of a list in various ways
```

```
# with the element's name
```

```
mylist$numbers
```

```
## [1] 10 11 12 13 14 15 16 17 18 19 20
```

```
mylist["numbers"]

## $numbers
## [1] 10 11 12 13 14 15 16 17 18 19 20
# via the index
mylist[1]

## $numbers
## [1] 10 11 12 13 14 15 16 17 18 19 20
# with [[]] we can access directly the content of the element
mylist[[1]]

## [1] 10 11 12 13 14 15 16 17 18 19 20
# lists can also be nested (list of lists of lists....)
mynestedlist <- list(a = mylist, b = 1:5)
```

Matrices and Data Frames

```
# matrices
mymatrix <- matrix(numeric_vector, nrow = 4)

## Warning in matrix(numeric_vector, nrow = 4): data length [11] is not a sub-multiple or multiple of
## the number of rows [4]
# get the second row
mymatrix[2,]

## [1] 11 15 19
# get the first two columns
mymatrix[, 1:2]

##      [,1] [,2]
## [1,]  10  14
## [2,]  11  15
## [3,]  12  16
## [4,]  13  17
# data frames ("lists as columns")
mydf <- data.frame(Name = c("Alice", "Betty", "Claire"), Age = c(20, 30, 45))
mydf

##      Name Age
## 1  Alice  20
## 2  Betty  30
## 3 Claire  45
# select the age column
mydf$Age

## [1] 20 30 45
mydf[, "Age"]

## [1] 20 30 45
```

```
mydf[, 2]

## [1] 20 30 45
# select the second row
mydf[2,]

##      Name Age
## 2 Betty  30
```

Classes and Data Structure

When importing data to R, using a new R-package, or extracting data from web sources, it is imperative to understand how the data is structured in the R-object (variable) one is working with. A lot of errors occur due to giving R functions input in the wrong format/structure. Having a close look at how the data stored in a variable that we want to work with further can help avoid such errors.¹ The function `class()` returns the class(es) of an R object:

```
# have a look at what kind of object you are dealing with
class(mydf)

## [1] "data.frame"
class(mymatrix)
```

```
## [1] "matrix" "array"
```

To get a clear idea of the structure of data stored in a specific variable, we can call the function `str()` (for structure) with the object of interest as the only function argument. This is particularly helpful when dealing with a complex/nested data structure.

```
# have a closer look at the data structure
str(mydf)

## 'data.frame':    3 obs. of  2 variables:
## $ Name: chr  "Alice" "Betty" "Claire"
## $ Age : num  20 30 45
```

¹Note that basic R functions as well as functions distributed via R packages are usually very well documented (look up documentation with `help(FUNCTION-NAME)` or `?FUNCTION-NAME`). An important part of such documentations is what class the input arguments of a function must have and what the class of the returned object is. See, for example the documentation of `mean()` by typing `?mean` into the R-console and hit enter.